Holy Grail Reference Document

Predicting Human Annotator Disagreement on CIFAR-10

3rd Year Deep Neural Networks Course Project

Purpose of this document: Complete technical reference for all project decisions, mathematics, architecture choices, experiments, and results. Every team member should be able to answer *any* viva question by consulting the relevant section.

Fixed random seed: `42` — Split: 6000/2000/2000

0 Contents

1 Introduction and Motivation

Standard deep learning treats classification as a problem of finding a single correct label for each image. When multiple annotators label the same image they often disagree: one says "cat", another says "dog". The conventional remedy is majority voting, which discards all disagreement as noise.

This project inverts that assumption: **disagreement is the signal**. We build a model that, given a 32×32 CIFAR-10 image, outputs a 10-dimensional probability distribution representing *how a population of approximately 50 human annotators would distribute their votes across the 10 CIFAR-10 classes.*

1.1 Why This Matters

1. **Generalization:** Peterson et al. (ICCV 2019) showed that training with soft human labels improves out-of-distribution accuracy and reduces cross-entropy on near-OOD test sets such as CIFAR-10.1 v4/v6 by up to 38%.

2. **Adversarial robustness:** Models trained on soft labels require roughly twice as many PGD iterations to fool, because they encode the perceptual similarity structure of the visual world.

3. **Semantic richness:** A model that confuses a dog with a cat is epistemically more sensible than one that confuses a dog with a truck. Soft labels capture this graded similarity.

Viva Answer Pointer

Q: Why not just train a normal classifier?
A: A normal classifier aims to recover the mode of $p(y|x)$. When $p(y|x)$ is multi-modal (genuine ambiguity), the mode discards information. Minimising cross-entropy with the full soft distribution is the maximum-likelihood objective for the *true* underlying label distribution, not an approximation of it.

2 Problem Statement and Formal Definitions

2.1 Task Definition

Symbol	Meaning
$x \in \mathbb{R}^{3\times32\times32}$	A CIFAR-10 image
$p(y\|x) \in \Delta^9$	Ground-truth annotator distribution (10-simplex) from CIFAR-10H
$q_\theta(y\|x) \in \Delta^9$	Model's predicted distribution
$\mathcal{Y} = \{0, \ldots, 9\}$	10 CIFAR-10 class indices
$N_a \approx 50$	Number of annotators per image

The objective is:

$$\min_\theta \ \mathbb{E}_{x\sim\mathcal{D}}[\mathcal{L}(p(y|x),\, q_\theta(y|x))] \tag{1}$$

where $\mathcal{L}$ is a distribution-matching loss (KL divergence, JSD, or our custom composite).

2.2 Key Mathematical Definitions

Key Formula

$$\text{Shannon Entropy:} \quad H(p) = -\sum_{y \in \mathcal{Y}} p(y) \log_2 p(y) \tag{2}$$

$$\text{Range: } 0 \leq H(p) \leq \log_2(10) \approx 3.3219 \text{ bits}$$

$$\text{KL Divergence:} \quad \mathrm{KL}(p \| q) = \sum_{y} p(y) \log \frac{p(y)}{q(y)} \tag{3}$$

$$\text{Note: } \mathrm{KL} \geq 0, \ \mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$$

$$\text{JSD:} \quad \mathrm{JSD}(p\|q) = \frac{1}{2}\mathrm{KL}\left(p \middle\| \frac{p+q}{2}\right) + \frac{1}{2}\mathrm{KL}\left(q \middle\| \frac{p+q}{2}\right) \tag{4}$$

$$\text{Range: } 0 \leq \mathrm{JSD} \leq 1, \ \text{symmetric, bounded}$$

$$\text{Cosine Similarity:} \quad \cos(p, q) = \frac{p \cdot q}{\|p\| \, \|q\|} \tag{5}$$

$$\text{Precision@}K: \quad P@K = \frac{\left|\mathcal{T}_K \cap \hat{\mathcal{T}}_K\right|}{K} \tag{6}$$

where $\mathcal{T}_K$ is the set of K images with highest true entropy and $\hat{\mathcal{T}}_K$ by predicted entropy.

Viva Answer Pointer

Q: What does high entropy mean for an image?
A: If $H(p) \approx 0$, all annotators agreed (e.g. unanimously "frog"). If $H(p) \approx 3.32$ bits, annotators were uniformly spread across all 10 classes—maximum possible confusion. In our dataset the mean entropy is **0.2228 bits** and the observed maximum is **2.8602 bits**, indicating that most images are clear but a long tail is genuinely ambiguous.

3 Dataset: CIFAR-10H

3.1 Data Sources

Dataset	Images	Labels	Role
CIFAR-10 (train)	50,000	Hard (one-hot)	Backbone pre-training only
CIFAR-10H (test set)	10,000	Soft (10-dim simplex)	Primary supervised signal

CIFAR-10H contains 511,400 crowdsourced judgements over the 10,000-image CIFAR-10 test set, collected via Amazon Mechanical Turk. Each image received ≈ 51 independent labels (range 47–63). Annotators scoring below 75% on attention-check trials were removed (14 of 2,571 total).

3.2 Data Splits

All splits used **stratified sampling** (preserving per-class ratio) with **fixed seed 42**.

Split	N	%	Use
Train	6,000	60%	Soft-label fine-tuning
Validation	2,000	20%	Early stopping, hyperparameter selection
Test	2,000	20%	Final held-out evaluation only

3.3 Dataset Statistics (Observed)

Statistic	Value
Total images	10,000
Soft label dimensionality	10
Majority vote agreement with hard label	99.2%
Mean Shannon entropy	0.2228 bits
Median Shannon entropy	≈0.05 bits
Observed maximum entropy	2.8602 bits
Theoretical maximum entropy	$\log_2(10) \approx 3.3219$ bits

Viva Answer Pointer

Q: Why is mean entropy so low (0.22 bits)?
A: Most CIFAR-10 images are unambiguous — annotators almost universally agree. The bulk of disagreement is concentrated in ≈30% of images, as also noted by Peterson et al. (2019). This creates a heavily right-skewed entropy distribution, which motivated our focal-weighting custom loss (Section 6.3).

3.4 Sanity Checks Applied

1. All soft-label vectors sum to 1.0 (tolerance 10^{-5}).
2. No NaN or Inf values in soft-label tensors.
3. Entropy values lie in $[0,\ \log_2(10) + 10^{-5}]$.
4. Hard labels lie in $\{0, \ldots, 9\}$.
5. Image tensor pixel values are in a reasonable normalised range.
6. No index overlap between splits.
7. Stratification: per-class counts are approximately equal across splits.

4 Data Augmentation

4.1 Policy

Augmentation was applied **only to the training split**. Validation and test images were passed through normalisation only, to prevent data leakage.

Transform	Train	Val/Test	Rationale
`ToTensor`	✓	✓	Convert PIL → float tensor
`Normalize` (μ, σ)	✓	✓	Zero-mean unit-var
`RandomHorizontalFlip`(p=0.5)	✓	–	Cheap invariance augmentation
`RandomCrop`(32, pad=4, reflect)	✓	–	Spatial invariance

Normalisation constants (CIFAR-10 channel statistics):

$$\mu = (0.4914,\ 0.4822,\ 0.4465), \qquad \sigma = (0.2023,\ 0.1994,\ 0.2010)$$

4.2 Why These Augmentations Only

Augmentations that change class semantics (e.g. rotation by 90°, colour jitter severe enough to change perceived class) were explicitly avoided. A 90° rotation turns a "ship" into something that looks like a "boat on its side"—possibly changing which annotators would still call it a "ship". Since our target is human perception, corrupting that perception in augmentation is counter-productive.

Viva Answer Pointer

Q: Does augmentation affect the soft labels?
A: No. The soft labels are fixed at the image level (empirically collected human votes). We apply augmentation only to the image tensor; the corresponding soft label is replicated unchanged. This is valid because horizontal flips and small crops do not meaningfully alter which class a human would assign to a 32×32 image.

5 Model Architecture

5.1 Overview

The model has two components:

1. **Backbone:** ResNet-18 adapted for 32×32 input.
2. **Prediction Head:** A linear or MLP layer mapping the 512-dimensional feature vector to a 10-dimensional simplex via `softmax`.

5.2 Backbone: ResNet-18 for CIFAR-10

Standard ImageNet-pretrained ResNet-18 uses a 7×7 convolution with stride 2 and a 3×3 max-pool at the stem, which aggressively downsamples the spatial resolution. Applied to

32×32 images this would reduce the spatial map to 4×4 after the stem alone—destroying useful local information.

We replaced the stem with a 3×3 convolution with stride 1, and **removed** the max-pool layer. This is the standard CIFAR adaptation of ResNet-18 used in nearly all CIFAR benchmarks.

5.2.1 Layer-by-Layer Representation

Layer	Output size	Operation	Notes
Input	$3 \times 32 \times 32$	—	RGB image
Stem (adapted)	$64 \times 32 \times 32$	Conv 3×3, stride 1, BN, ReLU	No max-pool
Layer 1	$64 \times 32 \times 32$	$2\times$ BasicBlock	2 residual blocks
Layer 2	$128 \times 16 \times 16$	$2\times$ BasicBlock, stride 2	Downsample
Layer 3	$256 \times 8 \times 8$	$2\times$ BasicBlock, stride 2	Downsample
Layer 4	$512 \times 4 \times 4$	$2\times$ BasicBlock, stride 2	Downsample
AvgPool	512	Global average pool	Spatial $\rightarrow$ vector
Head	10	Linear / MLP + Softmax	Distribution output

5.2.2 BasicBlock Definition

Each BasicBlock implements:

$$\mathbf{h} = \sigma(\mathrm{BN}(\mathbf{W}_2 * \sigma(\mathrm{BN}(\mathbf{W}_1 * \mathbf{x}))) + \mathbf{W}_s\mathbf{x}) \tag{7}$$

where $*$ denotes convolution, BN is Batch Normalisation, σ is ReLU, and $\mathbf{W}_s\mathbf{x}$ is the shortcut projection (identity if dimensions match, 1×1 conv otherwise).

5.3 Prediction Heads

5.3.1 Linear Head

$$q_\theta(y|x) = \mathrm{softmax}(\mathbf{W}\,\mathbf{z} + \mathbf{b})\,, \quad \mathbf{W} \in \mathbb{R}^{10\times 512},\ \mathbf{b} \in \mathbb{R}^{10} \tag{8}$$

Parameters: $512 \times 10 + 10 = 5{,}130$

Pros: Low capacity, low overfitting risk on 6,000 soft-label examples, directly interpretable weights as class-feature affinities.

Cons: Cannot capture non-linear interactions between backbone features and output classes.

5.3.2 MLP Head

$$\mathbf{h}_1 = \mathrm{ReLU}(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1), \quad q_\theta(y|x) = \mathrm{softmax}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2) \tag{9}$$

where $\mathbf{W}_1 \in \mathbb{R}^{256\times 512}$, $\mathbf{W}_2 \in \mathbb{R}^{10\times 256}$.

Parameters: $(512 \times 256 + 256) + (256 \times 10 + 10) = 131{,}330 + 2{,}570 = 133{,}902$

Pros: More expressive; can model non-linear feature-to-distribution mappings.

Cons: More parameters $\Rightarrow$ higher overfitting risk with small soft-label data; harder to interpret.

5.4 Parameter Count Summary

Component	Variant	Parameters	Total
ResNet-18 backbone	(CIFAR adapted)	11,173,952	—
Linear head	—	5,130	**≈11.18M**
MLP head	512→256→10	133,902	**≈11.31M**

Viva Answer Pointer

Q: Why ResNet-18 and not something bigger?
A: The soft-label training set has only 6,000 images. Larger models would overfit severely. ResNet-18 with ≈11M parameters is already likely over-parameterised for this data regime, which is why backbone initialisation strategy (pre-training) is critical (see Section 10). The goal of this project is not top-1 accuracy but distribution matching quality.

5.5 Output Constraint

The model's final layer is always a `softmax`, ensuring:

$$q_\theta(y|x) \geq 0 \quad \forall y, \qquad \sum_{y \in \mathcal{Y}} q_\theta(y|x) = 1$$

This is necessary because $p(y|x)$ is a probability distribution and all distribution- matching losses (KL, JSD) require the predicted output to also be a valid distribution.

6 Loss Function Design

6.1 Loss 1: KL Divergence (Mandatory Baseline)

Key Formula

$$\mathcal{L}_{\text{KL}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \text{KL}(p_i \| q_{\theta,i}) = \frac{1}{N} \sum_{i=1}^{N} \sum_{y} p_i(y) \log \frac{p_i(y)}{q_\theta(y|x_i)} \tag{10}$$

Plain English: Measures how many extra bits per symbol are needed if you use your model's distribution q to encode symbols actually drawn from the true distribution p. Equivalently, it is the expected log-likelihood ratio under p.

Suitability: KL is the natural loss for this task: minimising $\text{KL}(p\|q)$ is equivalent to maximum-likelihood estimation when p is the data distribution. Cross-entropy $\text{H}(p, q) = H(p) + \text{KL}(p\|q)$; since $H(p)$ does not depend on θ, minimising KL is equivalent to minimising cross-entropy.

Caveat: $\text{KL}(p\|q) = \infty$ if $q(y) = 0$ wherever $p(y) > 0$. We avoid this by clamping predictions: $q \leftarrow \max(q, 10^{-9})$. KL is also asymmetric and unbounded, making magnitude comparisons across models harder.

6.2 Loss 2: Jensen-Shannon Divergence

Key Formula

$$M_i = \frac{p_i + q_{\theta,i}}{2} \tag{11}$$

$$\mathcal{L}_{\text{JSD}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{2}[\text{KL}(p_i\|M_i) + \text{KL}(q_{\theta,i}\|M_i)] \tag{12}$$

Plain English: JSD measures the information gain from knowing which distribution a sample came from, using their mixture as a reference.

Why it is better than raw KL for training:

- Symmetric: $\text{JSD}(p\|q) = \text{JSD}(q\|p)$.
- Bounded: $\text{JSD} \in [0, 1]$ (when using log base 2).
- Never infinite even when support mismatches exist (mixture M always has full support wherever either p or q does).
- Smoother gradient landscape near $p = q$.

6.3 Loss 3: Custom Composite Loss (Focal-Weighted KL + Entropy Error Penalty)

Key Formula

$$\mathcal{L}_{\text{custom}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \underbrace{w_i \cdot \text{KL}(p_i\|q_{\theta,i})}_{\text{focal-weighted distribution matching}} + \lambda \cdot \underbrace{\frac{1}{N}\sum_{i=1}^{N}\left(H(q_{\theta,i}) - H(p_i)\right)^2}_{\text{entropy error penalty}} \tag{13}$$

where the focal weight is:

$$w_i = 1 + \alpha \cdot H(p_i), \quad \alpha = 2.0, \quad \lambda = 0.5 \tag{14}$$

Motivation: Two complementary observations drove this design:

1. **Entropy imbalance:** The training set is dominated by near-zero-entropy images (easy, unanimous agreement). Unweighted KL treats each image equally, so the gradient is overwhelmingly determined by easy images. The focal weight $w_i = 1 + 2H(p_i)$ up-weights high-disagreement images, forcing the model to learn the structure of ambiguous cases, which are more informative and harder.

2. **Entropy calibration:** KL minimisation does not explicitly penalise entropy misestimation. A model could achieve low KL by placing probability mass in the correct classes but with wrong spread. The entropy error term $\left(H(\hat{q}) - H(p)\right)^2$ directly penalises cases where the model's predicted uncertainty level differs from the true uncertainty level.

What it captures that standard losses do not: Both KL and JSD measure distributional distance but are agnostic to whether the model correctly identifies *how ambiguous* an image is. Our loss explicitly rewards getting the uncertainty magnitude right, which is crucial for Precision@K metrics.

Viva Answer Pointer

Q: Isn't the entropy penalty redundant with KL?
A: No. Two distributions can have identical KL divergence from a reference but very different entropies. Example: $p = [0.5, 0.5, 0, \ldots]$ and $q_1 = [0.45, 0.45, 0.05, 0.05, \ldots]$ vs $q_2 = [0.6, 0.4, 0, \ldots]$ may have similar KL from p but $H(q_1) > H(q_2)$. If $H(p) = 1$ bit, q_2 is better calibrated in entropy and our penalty selects it.

6.4 Bonus: Earth Mover's Distance (EMD)

Key Formula

$$\text{EMD}(p, q) = \min_{\gamma \in \Gamma(p,q)} \sum_{y,y'} \gamma(y, y') \cdot d(y, y') \tag{15}$$

where $\Gamma(p, q)$ is the set of joint distributions with marginals p and q, and $d(y, y')$ is the class-distance matrix (constructed using the WordNet semantic hierarchy for CIFAR-10 classes, e.g. cat-dog distance $<$ cat-truck distance).

EMD accounts for the *semantic structure* between classes: mistaking a cat for a dog is a smaller error than mistaking it for an airplane. KL and JSD treat all class confusions equally.

Limitation: Computationally expensive ($O(C^3 \log C)$ for C classes with generic solvers). With $C = 10$ this is tractable but adds training time.

6.5 Loss Function Comparison Summary

Loss	Symmetric	Bounded	Entropy-aware	Semantic
KL Divergence	×	×	×	×
JSD	✓	✓	×	×
Custom (ours)	×	×	✓	×
EMD (bonus)	✓	✓	×	✓

7 Training Protocol

7.1 Backbone Initialisation Strategies

We investigated three strategies (see Ablation A, Section 10):

1. **Random:** Xavier-uniform initialisation for all layers. Advantage: no prior bias. Disadvantage: requires learning all visual features from only 6,000 images—insufficient for stable convergence.

2. **CIFAR-10 hard-label pre-training:** First train on all 50,000 CIFAR-10 training images with cross-entropy on one-hot labels; then fine-tune backbone + head on soft labels. Advantage: backbone learns rich CIFAR-relevant features. Disadvantage: one-hot pretraining instils a "peak-confidence" prior that must be unlearned during fine-tuning.

3. **ImageNet pre-training:** Use torchvision ResNet-18 pretrained on ImageNet. Advantage: strongest generalised visual features. Disadvantage: domain gap (ImageNet vs. 32×32 CIFAR images).

7.2 Two-Stage Training (Recommended)

Algorithm 1 Two-Stage Training Pipeline

Stage 1 (Pretraining): Train ResNet-18 on CIFAR-10 (50k images) with hard labels
  Optimiser: Adam, lr $= 0.1$, weight decay $= 5 \times 10^{-4}$
  Epochs: 100, batch size: 128
  Loss: $\mathcal{L}_{\text{CE}} = -\sum_y \delta_{y,\hat{y}} \log q(y|x)$
Stage 2 (Fine-tuning): Fine-tune on CIFAR-10H (6k soft-label images)
  Optimiser: Adam, lr $= 10^{-4}$ (reduced)
  Loss: KL, JSD, or Custom (one run per loss)
  Early stopping: patience $= 15$ epochs on validation loss
  Save best checkpoint by minimum validation KL divergence

7.3 Fixed Hyperparameters (All Runs)

Hyperparameter	Value
Optimiser	Adam
Weight decay	5×10^{-4}
Learning rate schedule	Cosine annealing to 10^{-6}
Batch size	128
Augmentation	HorizontalFlip + RandomCrop (train only)
Random seed	42
Early stopping patience	15 epochs
Metric for early stopping	Validation KL divergence

Viva Answer Pointer

Q: Why Adam and not SGD?
A: With 6,000 training samples per epoch the gradient noise is substantial, especially in the fine-tuning stage where updates are small. Adam's adaptive per-parameter learning rates make it more robust to sparse or heterogeneous gradients—particularly relevant when the distribution targets vary greatly in entropy (uniform vs. highly peaked).

7.4 Logging

All runs logged per epoch:

- Training loss (the chosen $\mathcal{L}$)
- Validation loss (same $\mathcal{L}$)
- Validation KL divergence (common reference metric across all runs)
- Validation cosine similarity
- Best checkpoint saved whenever validation KL improves

8 Experiments Run

8.1 Experiment 1: Core Performance Evaluation

What we did: Evaluated our best trained model (for each loss function) on the held-out test set of 2,000 images using all required metrics.

Metrics computed:

1. Distribution matching: mean $\pm$ std of KL divergence, JSD, cosine similarity.
2. Entropy prediction quality: Pearson r and Spearman ρ between $H(p_i)$ and $H(q_{\theta,i})$ across 2,000 test images.
3. Precision@K for $K \in \{100, 200, 500\}$.

Expected: Models trained with soft-label losses should better capture the shape of human disagreement than a random baseline. The custom loss should show improved Precision@K due to focal weighting.

8.2 Experiment 2: Loss Function Comparison

What we did: Trained three models (KL loss, JSD loss, Custom loss), held backbone and head architecture fixed (ResNet-18 + Linear head, ImageNet pre-training), varied only the loss function.

Expected: KL and JSD should produce similar distribution-matching quality (both minimise distributional distance) but JSD may be more stable due to boundedness. Custom loss should improve Precision@K but possibly at some cost to raw KL divergence on test set.

8.3 Experiment 3: Backbone Initialisation Ablation (Ablation A)

What we did: Trained three versions of the model (same: JSD loss + Linear head) under three backbone init strategies: Random, CIFAR-10 pretrained, ImageNet pretrained.

Expected: ImageNet > CIFAR-10 pretrained > Random in terms of final test KL, because stronger initialisation provides better features. However, the ImageNet gap may be small since CIFAR-10 images are very small resolution.

8.4 Experiment 4: Head Architecture Ablation (Ablation D)

What we did: Compared Linear vs. MLP head under identical backbone (ResNet-18, CIFAR-10 pretrained) and loss (KL).

Expected: Linear head may generalise better given the small data regime. MLP head may overfit, producing lower training loss but worse test metrics.

8.5 Experiment 5: Training Data Strategy (Ablation C)

What we did: Compared (a) soft-label training only (from random init) vs. (b) hard-label pretraining then soft-label fine-tuning.

Expected: Strategy (b) should substantially outperform (a) because 6,000 images are insufficient to learn meaningful visual representations from scratch.

8.6 Experiment 6: Robustness Check — OOD Corruptions (Robustness B)

What we did: Applied three corruption types (Gaussian noise, Gaussian blur, contrast reduction) at five severity levels (1 = mild, 5 = severe) to the 2,000 test images. Measured how mean predicted entropy $\mathbb{E}[H(q_\theta(y|x^c))]$ changes with severity.

Corruption functions implemented:

$$\text{Gaussian noise:} \quad x^c = \text{clip}(x + \mathcal{N}(0, (s \cdot 0.05)^2 \mathbf{I})) \tag{16}$$

$$\text{Gaussian blur:} \quad x^c = \text{GaussianBlur}(x,\ k = 2s + 1,\ \sigma = 0.5s) \tag{17}$$

$$\text{Contrast reduction:} \quad x^c = \text{AdjustContrast}(x,\ \text{factor} = \max(0.1, 1 - 0.15s)) \tag{18}$$

Expected: Increasing corruption severity should increase predicted entropy (model becomes more uncertain). If entropy does not monotonically increase, it indicates the model has learned corrupt-invariant features or that the backbone is too uncertain even on clean images.

8.7 Experiment 7: Class-Conditional Performance (Robustness C)

What we did: Partitioned test images by their majority-vote class label and computed per-class KL divergence, JSD, and cosine similarity.

Expected: Classes that are semantically close (cat/dog, deer/horse) should show higher baseline entropy and harder distribution matching. Classes with clear visual distinctiveness (ship, frog) should have lower entropy and easier matching.

8.8 Experiment 8: Grad-CAM Analysis (Explainability)

What we did: Ran Grad-CAM on the final residual block (`layer4[-1]`) of ResNet-18 using three strategies:

- `top_pred`: Gradient with respect to the argmax of q_θ.
- `top_true`: Gradient with respect to the dominant class in p.

- `entropy_weighted`: Weighted sum of per-class CAMs using $w_y = -p(y)\log p(y)$ (attention weighted by contribution to entropy).

Grad-CAM formula:

$$L^c_{\text{GradCAM}} = \text{ReLU}\left(\sum_k \alpha^c_k A^k\right), \quad \alpha^c_k = \frac{1}{Z}\sum_{i,j}\frac{\partial S^c}{\partial A^k_{ij}} \tag{19}$$

where A^k is the k-th feature map at the target layer and S^c is the score for class c.

Expected: Low-entropy images should show tight, focused heatmaps on the dominant object. High-entropy images should show diffuse heatmaps with activation spread across multiple possible objects.

8.9 Experiment 9: Failure Case Analysis (Explainability)

What we did: Sorted all 2,000 test images by $\text{KL}(p_i \| q_{\theta,i})$ and inspected the 10 worst cases. For each failure case reported: image, true distribution p, predicted distribution q, $H(p)$, $H(q)$, and a hypothesis for the failure mode.

Hypothesised failure modes:

- **Type 1: Collapsed prediction on ambiguous image.** $H(p) \gg H(q)$: model is over-confident where humans are uncertain.
- **Type 2: Spread prediction on clear image.** $H(p) \approx 0$ but $H(q) > 1$: model is uncertain where humans are not.
- **Type 3: Wrong modal class.** $\arg\max q \neq \arg\max p$: model identifies the wrong dominant class.

8.10 Experiment 10: Annotator Subsampling (Robustness A)

What we did: Recomputed soft labels using subsets of annotators ($N_a \in \{5, 10, 20, 50\}$) and evaluated how model predictions correlate with these noisier ground truths.

Expected: With fewer annotators the empirical distribution is noisier; the model's predictions, trained on full-annotator distributions, should remain more stable than the subsampled targets.

9 Results

Note: This section contains *placeholder* rows. Replace with your actual computed values before the viva. The table structure, metric names, and interpretation guidance are complete.

9.1 Core Metrics Table

Model	KL↓	JSD↓	Cos↑	Pearson↑	Spearman↑	P@100↑	P@200↑	P@5
KL Loss (Linear)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
JSD Loss (Linear)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Custom Loss (Linear)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Custom Loss (MLP)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Random baseline	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*

How to interpret the metrics:

- **KL ≈ 0**: predicted distribution is almost identical to true.
- **Pearson/Spearman close to 1**: model correctly ranks images by uncertainty level.
- **P@500 ¿ 0.5**: model identifies at least half of the 500 most ambiguous images.
- **Cosine ≈ 1**: distributions point in the same direction in probability space.

9.2 Training and Validation Loss Curves

Training and validation loss curves for each loss function should show:

- **Convergence:** Validation loss decreasing and plateauing (not diverging), confirming that early stopping is working correctly.
- **Generalisation gap:** Train loss < Val loss is expected; a large gap indicates overfitting. With only 6,000 training images this gap may be visible but should be controlled by early stopping.

9.3 Entropy Scatter Plot Observations

The scatter plot of $H(p_i)$ (x-axis) vs. $H(q_{\theta,i})$ (y-axis) should ideally lie along the diagonal $y = x$. Common failure patterns:

- **Regression to mean:** Points form a horizontal band at the mean entropy—model outputs similar entropy for all images regardless of true uncertainty.
- **Under-prediction of high entropy:** Points below the diagonal at high $H(p)$—model does not predict extreme ambiguity.
- **Good model:** Approximately diagonal scatter with Pearson $r > 0.7$.

10 Ablation Studies

We completed three compulsory ablations: **A** (Backbone Initialisation), **C** (Training Data Strategy), and **D** (Prediction Head Architecture). Ablation B (Loss Comparison) is reported as part of the core evaluation.

10.1 Ablation A: Backbone Initialisation

Init Strategy	KL↓	JSD↓	Pearson↑	P@500↑
Random	*TBD*	*TBD*	*TBD*	*TBD*
CIFAR-10 pretrained	*TBD*	*TBD*	*TBD*	*TBD*
ImageNet pretrained	*TBD*	*TBD*	*TBD*	*TBD*

Expected finding: Random initialisation should perform worst because 6,000 training images are insufficient to learn feature representations from scratch. Pre-training, whether on CIFAR-10 or ImageNet, provides a feature extractor that already separates the 10 classes in representation space. The fine-tuning stage then shifts probability mass to match the human distribution pattern.

Causal explanation: A randomly initialised network outputs near-uniform distributions for all images early in training (before features are learned). This means the gradient signal for high-entropy images (which have near-uniform p) is near zero—the model cannot distinguish between the few images it should predict high entropy for and all others. A pre-trained backbone produces distinctly different feature vectors for different image types, allowing the head to learn entropy-relevant patterns from the first epoch.

10.2 Ablation C: Training Data Strategy

Strategy	KL↓	Pearson↑	P@500↑
Soft-label only (random init)	*TBD*	*TBD*	*TBD*
Hard pretrain + soft fine-tune	*TBD*	*TBD*	*TBD*

Causal explanation: The CIFAR-10H dataset has only 6,000 soft-label examples. This is enough to adjust a *pre-existing* good feature extractor to produce better-calibrated distributions, but not enough to *learn* a feature extractor. The 50,000 hard-label images fill this gap despite providing only one-hot targets: the backbone learns class-discriminative features, which generalise to the softer distribution-prediction task.

10.3 Ablation D: Prediction Head Architecture

Head	Params	KL↓	JSD↓	P@500↑
Linear (512→10)	5,130	*TBD*	*TBD*	*TBD*
MLP (512→256→10)	133,902	*TBD*	*TBD*	*TBD*

Expected finding: With only 6,000 soft-label images, the MLP head is likely to overfit. The linear head generalises better because the backbone features are already rich and the mapping from features to distribution is approximately linear (the backbone does the heavy lifting).

11 Soft Label vs. Hard Label: Conceptual Comparison

11.1 Definitions

Property	**Hard Label**	**Soft Label**
Representation	$y \in \{0, \ldots, 9\}$	$p \in \Delta^9$
Example	$\hat{y} = 3$ (cat)	$[0, 0, 0, 0.6, 0, 0.3, 0, 0, 0, 0.1]$
Entropy	0 bits (always)	Variable: 0 to $\log_2(10)$ bits
Loss used with	Cross-entropy (one-hot)	KL, JSD, CE with soft targets
Dataset	CIFAR-10 (50k)	CIFAR-10H (10k)
Source	Human modal choice	Full human vote distribution

11.2 Why Hard Labels Are Insufficient for This Task

Training exclusively on hard labels instils a single-mode prior. Consider the optimal output of a hard-label-trained classifier for image x:

$$q^*_{\text{hard}}(y|x) = \delta_{y,\hat{y}^*}$$

i.e. all probability mass on the majority-vote class. By contrast, the true annotator distribution for the same image might be:

$$p(y|x) = [0, 0, 0, 0.6, 0, 0.3, 0, 0, 0, 0.1] \quad \text{(cat 60\%, dog 30\%, deer 10\%)}$$

A hard-label-trained model cannot produce such a distribution—its softmax would be sharply peaked. Peterson et al. (ICCV 2019) demonstrate empirically that hard-label-trained CNNs exhibit poor second-best accuracy (SBA) and systematically over-confident predictions on ambiguous images.

11.3 Role of Hard Labels in Our Pipeline

Despite not being the primary target, hard labels play an important role:

1. **Backbone pre-training (Stage 1):** 50,000 hard-label images teach the backbone to produce class-discriminative features. Without this, the backbone cannot separate the 10 classes.

2. **They must not be used as soft targets:** Hard labels correspond to $p(y|x) = \delta_{y,\hat{y}}$, which are the most extreme possible soft labels. Using them as soft targets during fine-tuning would force the model away from the genuine human distribution.

11.4 Comparison of Training Regimes (from Peterson et al. 2019)

The original CIFAR-10H paper compared (crossentropy lower = better):

Training scheme	**Cross-entropy on CIFAR-10H**	**Generalisation trend**
Hard labels (pretrained only)	∼0.60	Worst
mixup	∼0.36–0.47	Better
Category soft targets	∼0.33–0.44	Good
Human soft targets (full)	∼0.26–0.35	Best

Viva Answer Pointer

Q: If hard-label training is insufficient, why do we bother with it at all? A: The 50,000 hard-label images give us something we cannot get from 6,000 soft-label images: a well-trained visual feature extractor. The two stages are complementary. Stage 1 answers "what does the image look like in representation space?"; Stage 2 answers "given where this image lives in representation space, what will humans say?".

12 Troubles and Challenges Encountered

12.1 Data Alignment Issue

Problem: CIFAR-10H provides soft labels for the CIFAR-10 *test* set (10,000 images). The standard `torchvision` CIFAR-10 download puts test images in a separate file. Naively loading with `train=True` and then trying to pair with CIFAR-10H labels would silently misalign images and soft labels.

Solution: We always load with `train=False` in the dataset wrapper and verified alignment by checking that the hard-label majority vote from the soft distribution matches the CIFAR-10 test hard labels for >99% of images (observed: 99.2% agreement).

12.2 Entropy Distribution Imbalance

Problem: The soft-label dataset is heavily skewed: ≈70% of images have entropy < 0.1 bits (near-unanimous agreement). Standard KL loss focuses training on this dominant group, potentially learning to output low-entropy distributions for everything.

Solution: Our custom focal-weighted loss (Eq. 13) addresses this by scaling each image's loss contribution by $1 + 2H(p_i)$. We also monitored the entropy scatter plot throughout training to detect regression-to-mean failures early.

12.3 Overfitting on Small Soft-Label Data

Problem: 6,000 images with a model that has 11M parameters leads to severe overfitting. We observed training loss decreasing but validation KL divergence increasing after ≈10–20 fine-tuning epochs.

Solution: Early stopping with patience=15 on validation KL. Using the pre-trained backbone (rather than random init) meant that the meaningful degrees of freedom were effectively much smaller (only the head needed substantial learning).

12.4 Softmax Numerical Stability

Problem: KL divergence is undefined when $q(y) = 0$ for any y with $p(y) > 0$. Even with softmax, extreme pre-softmax logits can produce near-zero probabilities ($< 10^{-38}$ in float32).

Solution: Clamped all predicted probabilities to a minimum of 10^{-9} before computing any logarithm. Used PyTorch's `F.kl_div` in log-space mode to avoid explicit log-of-small-number issues.

12.5 Grad-CAM Target Choice for Soft-Label Models

Problem: Standard Grad-CAM is designed for single-class classifiers where the target class is unambiguous. For soft-label models, which class should we compute gradients with respect to?

Solution: Implemented three strategies (see Section 7). The `entropy_weighted` strategy, which computes a weighted sum of per-class CAMs using $w_y = -p(y) \log p(y)$, turned out to be the most informative for high-entropy images because it highlights regions contributing to *all* confused classes, not just the dominant one.

12.6 Windows Multiprocessing Limitation

Problem: `DataLoader` with `num_workers > 0` crashes on Windows due to multiprocessing constraints.

Solution: Set `num_workers=0` as the default. This slows data loading but removes all crashes. On Linux/Mac systems, `num_workers=4` was used for faster training.

13 Explainability Analysis

13.1 Grad-CAM: Low vs. High Entropy Images

Low-entropy images (e.g. a clear frog on a green background, $H(p) < 0.1$):

- Grad-CAM heatmap is tightly focused on the dominant object.
- The model attends to discriminative local features (texture, shape).
- All three strategies (`top_pred`, `top_true`, `entropy_weighted`) produce similar heatmaps because the dominant class is overwhelmingly predicted.

High-entropy images (e.g. a blurry cat/dog silhouette, $H(p) > 1.5$):

- Grad-CAM heatmap under `entropy_weighted` is diffuse across the whole image.
- The model has no single region that drives its prediction.
- Under `top_pred`, the heatmap may focus on an arbitrary local feature that pushed the model toward its modal prediction—often not interpretable as the semantically relevant region.

13.2 Manual Disagreement Source Categorisation

High-entropy images were manually inspected and categorised:

Category	Description
Ambiguous object identity	Image could plausibly be multiple classes (cat vs. dog silhouette)
Poor image quality	Blurry, overexposed, or very low contrast
Multi-object content	Two different objects from different classes visible
Boundary case	Object is between two classes (foal ≈ horse/deer)
Unusual viewpoint	Angle makes identification difficult

14 Robustness Analysis

14.1 Corruption Experiment Results

Expected behaviour: Mean predicted entropy should increase monotonically with corruption severity for all three corruption types.

If entropy does not increase: This may indicate that the model has learned globally high-entropy outputs regardless of input (regression to mean), or that the backbone representations are robust to the specific corruption type applied. This should be analysed, not ignored.

Gaussian noise corrupts local texture, which is important for fine-grained class distinctions (e.g. distinguishing cat from dog). Expected to be the most disruptive to entropy.

Gaussian blur removes high-frequency detail. For low-resolution 32×32 images, even mild blurring can destroy most of the class-discriminative information.

Contrast reduction reduces signal-to-noise ratio. At extreme severity, images approach uniform grey.

14.2 Class-Conditional Analysis

Expected findings:

- **High baseline entropy classes:** Cat ($\leftrightarrow$ dog), Deer ($\leftrightarrow$ horse), Bird ($\leftrightarrow$ airplane/-ship). The model should perform worse on these (higher KL, lower cosine sim).
- **Low baseline entropy classes:** Ship, Frog, Automobile, Truck. These are visually distinctive; the model should perform well.

15 Concluding Notes

15.1 Summary of Technical Contributions

1. **Soft-label prediction pipeline:** Complete, reproducible pipeline from data loading through to test evaluation, with fixed seed and stratified splits.
2. **Adapted ResNet-18:** CIFAR-compatible backbone with removed max-pool and modified stem.
3. **Custom composite loss:** Focal-weighted KL with entropy calibration penalty, motivated by the entropy imbalance in CIFAR-10H.
4. **Comprehensive evaluation:** Eight metrics spanning distribution matching quality, entropy calibration, and ranking.
5. **Explainability:** Entropy-weighted Grad-CAM strategy tailored for soft-label models.

15.2 Key Takeaways for Viva

Viva Answer Pointer

1. The model does **not** classify; it **predicts a distribution**.
2. KL divergence is the natural loss but JSD is more stable; our custom loss additionally calibrates entropy.
3. Pre-training is **essential** because 6,000 samples are too few to learn visual features from scratch.
4. High-entropy images are harder, rarer, and more informative—the focal weighting addresses this.
5. Grad-CAM for soft-label models requires a deliberate choice of target class; entropy-weighted CAM is most informative for ambiguous images.
6. Failure cases fall into three types: collapsed predictions, diffuse predictions, or wrong-modal predictions.
7. Soft labels improve generalisation and adversarial robustness (Peterson et al. 2019)—our model is in the same framework.

15.3 What We Would Do Differently

- **Temperature scaling post-hoc:** A calibration step after training (learning a single scalar T such that $q_T(y|x) \propto q(y|x)^{1/T}$) could further improve entropy calibration without retraining.
- **Mixup in soft-label regime:** Mixing two images with their soft labels: $\tilde{p} = \lambda p_i + (1-\lambda)p_j$, which creates richer training signal.
- **Larger model with stronger regularisation:** DropOut + weight decay on the head could allow a deeper backbone with the small data.

16 Reference: File-by-File Codebase Map

File	Purpose
`training/config.py`	All hyperparameters, paths, seed (FIXED_SEED=42)
`data/dataset.py`	`CIFAR10HWrapper`: image + soft label loading
`data/pipeline.py`	Transforms, splits (stratified), DataLoaders, sanity checks
`data/statistics.py`	Entropy, confusion matrix, class-wise stats
`models/cifar_resnet.py`	ResNet-18 with CIFAR stem + prediction heads
`losses/kl_divergence.py`	$\mathcal{L}_{\text{KL}}$
`losses/js_divergence.py`	$\mathcal{L}_{\text{JSD}}$
`losses/custom_losses.py`	Focal-weighted KL + entropy penalty
`losses/emd_loss.py`	Earth Mover's Distance (bonus)
`training/train.py`	Main training loop with early stopping
`training/metrics_logger.py`	CSV/NPY logging of per-epoch metrics
`evaluation/metrics.py`	All test metrics (KL, JSD, cosine, Pearson, P@K)
`evaluation/eval.py`	Full test-set evaluation loop
`evaluation/robustness.py`	Corruption experiments
`evaluation/visualize.py`	Entropy scatter, metric comparison, qualitative grid
`explainability/grad_cam.py`	GradCAM class + plotting functions
`visualisations/training_curves.py`	Train/val loss curve plotting
`visualisations/grad_cam_plots.py`	Grad-CAM grid + entropy vs. CAM plots
`main_data.py`	Orchestrates full data pipeline (7-step)
`abalation.py`	CSV aggregation across ablation runs

This document was compiled from the full project codebase and assignment brief.
Fill in all `TBD` entries with actual experimental results before the viva.